```
    result = run()
    reset()
    result

UP = 38
DOWN = 40
Q = 81
A = 65
```

**Some variables (uppercase to indicate constants) that map the keycode supplied by an event to the key on the keyboard that was pressed.**

```
doc =
  on: (event, fn) ->
    old = document["on#{event}"] || ->
    document["on#{event}"] = (e) ->
      old e
      fn e
```

**A thin wrapper around the document used to attach listeners to keyboard events.**

```
class Paddle

  constructor: (@top=0, @left=0) ->
    @render()

  move: (displacement) ->
    @top += displacement*5
    @paddle.style.top = @top + 'px'
```

**Moves the graphical paddle on the screen.**

```
  render: ->
    @paddle = document.createElement 'div'
    @paddle.className = 'paddle'
    @paddle.style.backgroundColor = 'black'
    @paddle.style.position = 'absolute'
    @paddle.style.top = "#{@top}px"
    @paddle.style.left = "#{@left}px"
    @paddle.style.width = '20px'
    @paddle.style.height = '100px'
    document.querySelector('#pong').appendChild @paddle
    displacement = ([up,down]) ->
      (event) ->
        switch event.keyCode
          when up then -1
          when down then 1
          else 0

    move = (paddle) ->
      (moves) ->
        for displacement in moves
          paddle.move displacement

    keys = (expected) ->
      (pressed) ->
        pressed.keyCode in expected
```

**Creates a graphical paddle on the screen.**

```
    paddle1 = new Paddle 0,0
    paddle1.keys = [Q,A]

    paddle2 = new Paddle 0,200
    paddle2.keys = [UP,DOWN]
```

**Paddle I is controlled by Q and A keys; paddle 2 is controlled by up- and down-arrow keys.**

```
    withEvents(doc, 'keydown')
    .filter(keys paddle1.keys)
    .map(displacement paddle1.keys)
    .drain(move paddle1)

    withEvents(doc, 'keydown')
    .filter(keys paddle2.keys)
    .map(displacement paddle2.keys)
    .drain(move paddle2)
```

**The core movement logic becomes clear.**